

SOFTWARE REQUIREMENTS SPECIFICATION

Task Management App

(Trello Clone – MERN Stack Web Application)

Prepared by : padhiyar Jatin D. (IT, SPCET)

Date : July 03, 2026

Table of Contents

1. Introduction.....	4
1.1 Purpose	4
1.2 Project Scope	4
2. Overall Description.....	5
2.1 Product Perspective.....	5
2.2 Product Features.....	5
2.3 User Classes and Characteristics.....	5
2.4 Operating Environment	6
2.5 Design and Implementation Constraints	6
2.6 User Documentation.....	6
3. System Features	7
3.1 User Features	7
3.1.1 Registration	7
3.1.2 Login	7
3.1.3 Dashboard	7
3.1.4 Create Task	7
3.1.5 View Tasks.....	7
3.1.6 Update Task	7
3.1.7 Delete Task	7
3.1.8 Update Status	7
3.1.9 Search & Filter.....	7
3.1.10 Due Date Management	8
3.1.11 Logout	8
3.2 Admin Features.....	8
3.2.1 Login	8
3.2.2 Dashboard	8
3.2.3 Monitor Users.....	8
3.2.4 Monitor Tasks	8
4. External Interface Requirements	9
4.1 User Interface.....	9
4.2 Software Interface.....	9
4.3 Communication Interface	9
5. Other Non-Functional Requirements.....	10
5.1 Performance Requirements	10
5.2 Security Requirements	10
5.3 Reliability.....	10
5.4 Availability	10
5.5 Scalability	10
5.6 Software Quality Attributes.....	11

6. Other Requirements	12
Appendix A: Glossary	12
Appendix B: Analysis Models	13

1. Introduction

1.1 Purpose

- The Task Management App is a web-based application developed using the MERN Stack. It helps users organize and manage their daily tasks efficiently. Users can securely register, log in, create tasks, update task status, set due dates, and monitor their progress through a dashboard. The application is designed to improve productivity while demonstrating modern full-stack web development concepts.

1.2 Project Scope

- The Task Management App provides a secure platform for users to manage personal tasks from anywhere using a web browser.

The system aims to:

- Allow users to register and log in securely.
- Create, update, and delete tasks.
- Manage task status.
- Set due dates for tasks.
- Display task statistics on the dashboard.
- Provide a responsive and user-friendly interface.
- Ensure secure data storage using JWT authentication.

2. Overall Description

2.1 Product Perspective

- The Task Management App is a standalone web application that helps users organize their daily work. The application consists of three main components:
 - React.js Frontend
 - Express.js & Node.js Backend
 - MongoDB Database
- The frontend communicates with the backend using REST APIs, while MongoDB stores all user and task information securely. Each user can only access and manage their own tasks.

2.2 Product Features

- At a high level, the system provides the following major feature groups:
 - Secure user authentication using JWT and bcrypt-based password hashing
 - A personalized dashboard summarizing task statistics
 - Complete task lifecycle management (create, read, update, delete, status change)
 - Due date assignment and overdue task tracking
 - Search and multi-criteria filtering of tasks
 - A minimal administrative view for monitoring users and tasks

2.3 User Classes and Characteristics

- Guest User
 - Can register a new account.
 - Can log in using valid credentials.
 - Cannot access the dashboard without authentication.
- Registered User
 - Can create, edit, update, and delete tasks.
 - Can view personal dashboard statistics.
 - Can search and filter tasks.
 - Can update task status.
 - Can securely log out.

2.4 Operating Environment

- The application is designed to operate in the following environment:
 - Client Side: Any modern web browser (Chrome, Firefox, Edge, Safari) on Windows, Linux, macOS, or mobile devices
 - Server Side: Node.js runtime environment hosted on a Windows or Linux server
 - Database: MongoDB Atlas (cloud-hosted) or local MongoDB instance
 - Network: Requires an active internet connection for client-server communication

2.5 Design and Implementation Constraints

- Authentication must be implemented using JSON Web Tokens (JWT)
- Passwords must be hashed using bcrypt prior to storage; plain-text passwords must never be stored
- All API endpoints must follow RESTful design conventions
- The user interface must be built with React.js and styled using Tailwind CSS
- The application must be responsive and function correctly on both desktop and mobile screen sizes

2.6 User Documentation

- The system will provide simple and easy-to-understand documentation for users. It will include: User Registration Guide , Login Guide , Task Management Guide , Dashboard Overview , Frequently Asked Questions (FAQ). The documentation will be available in digital format.

3. System Features

3.1 User Features

3.1.1 Registration

Allows a new individual to create a personal account by providing a name, email address, and password.

3.1.2 Login

Enables a registered user to authenticate using their email and password.

3.1.3 Dashboard

Presents a summary view of the user's task activity immediately after login, giving a quick snapshot of overall progress.

3.1.4 Create Task

Allows the user to add a new task with a title, description, priority level, status, and optional due date.

3.1.5 View Tasks

Displays all tasks belonging to the authenticated user in a list or board-style layout, showing key details such as title, status, priority, and due date.

3.1.6 Update Task

Allows the user to modify the details of an existing task, including title, description, priority, and due date.

3.1.7 Delete Task

Permits the user to permanently remove a task from their task list after confirmation.

3.1.8 Update Status

Enables the user to change a task's status (e.g., Pending, In Progress, Completed) without editing other task fields.

3.1.9 Search & Filter

Allows the user to quickly locate tasks by keyword search on the title, and to narrow the visible task list using status and priority filters.

3.1.10 Due Date Management

Allows the user to assign, modify, or remove a due date for a task, and provides visual indicators for tasks that are approaching or past their due date.

3.1.11 Logout

Allows the user to safely end their authenticated session.

3.2 Admin Features

3.2.1 Login

Allows the admin to authenticate using a designated admin account, following the same JWT-based authentication mechanism as regular users.

3.2.2 Dashboard

Provides the admin with a system-wide overview, distinct from the individual user dashboard.

3.2.3 Monitor Users

Allows the admin to view a read-only list of all registered users for oversight purposes.

3.2.4 Monitor Tasks

Allows the admin to view a read-only list of all tasks created across the system for oversight purposes, without edit or delete rights.

4. External Interface Requirements

4.1 User Interface

- The user interface will be developed using React.js with Tailwind CSS to ensure a clean, modern, and consistent visual design. The UI will include a registration page, login page, a dashboard with statistic cards, a task board/list view, task creation and edit forms (typically modal-based), and search/filter controls. The interface must be fully responsive, adapting seamlessly across desktop, tablet, and mobile screen sizes.

4.2 Software Interface

- React.js : Building the single-page application user interface
- Tailwind CSS : Responsive, utility-first UI styling
- Axios : Sending requests from the front-end to the back-end API
- Node.js / express.js : Handling API requests and business logic
- MongoDB with Mongoose : Persistent storage of user and task data
- JWT : Issuing and verifying secure session tokens
- Bcrypt : Hashing passwords before storage

4.3 Communication Interface

- The system uses standard web communication protocols.
 - Communication between frontend and backend is performed using **RESTful APIs**.
 - Data is exchanged in **JSON** format.
 - **HTTP/HTTPS** protocols are used for secure communication.
 - JWT tokens are included in protected API requests for authentication.

5. Other Non-Functional Requirements

5.1 Performance Requirements

- API responses for standard CRUD operations should complete within 2 seconds under normal load conditions.
- The dashboard must load and render task statistics within 3 seconds of successful login.
- The system should efficiently support at least 100 concurrent users without significant performance degradation.

5.2 Security Requirements

- All passwords must be hashed using bcrypt; plain-text passwords must never be stored or logged.
- All protected API routes must validate a signed JWT before granting access.
- Users must only be able to access, modify, or delete tasks that belong to their own account.
- Sensitive configuration values (database URIs, JWT secrets) must be stored in environment variables using dotenv and excluded from version control.

5.3 Reliability

- The system shall handle invalid input, network interruptions, and failed requests gracefully, returning clear and consistent error messages without crashing the application or exposing internal implementation details.

5.4 Availability

- The application targets high availability during standard usage hours when deployed on a hosted environment, with MongoDB Atlas providing managed database uptime and redundancy.

5.5 Scalability

- The system's modular REST API and document-based database schema allow for horizontal scaling and future feature expansion, such as team collaboration or additional task attributes, with minimal restructuring of the existing codebase.

5.6 Software Quality Attributes

- Usability: An intuitive, minimal-clutter interface suitable for first-time users
- Maintainability: Modular folder structure separating routes, controllers, models, and components
- Portability: Compatible with Windows and Linux environments and any modern web browser
- Testability: RESTful API endpoints independently verifiable using tools such as Postman
- Scalability : The application should easily support additional features and increasing users.

6. Other Requirements

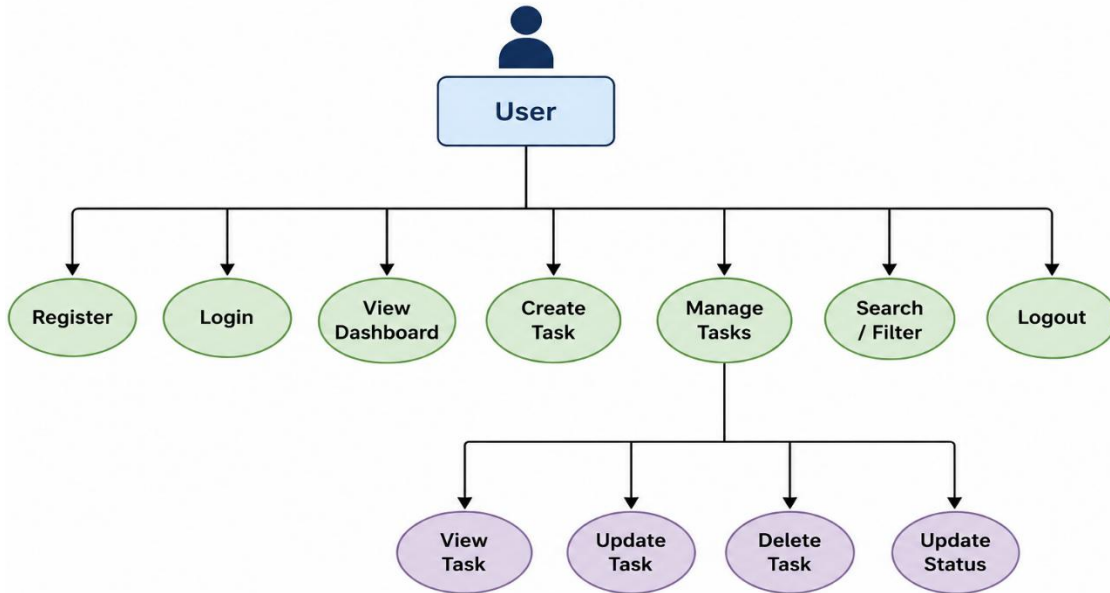
- The Task Management App should satisfy the following additional requirements:
 - The application should provide a responsive user interface for desktop, tablet, and mobile devices.
 - The system should validate all user inputs before storing data.
 - The application should maintain proper error logs for debugging.
 - The database should support backup and recovery mechanisms.
 - The project should follow a modular coding structure for easier maintenance.
 - The application should be designed to support future feature enhancements without major modifications.

Appendix A: Glossary

Term	Definition
MERN Stack	A web development stack comprising MongoDB, Express.js, React.js, and Node.js
JWT	JSON Web Token — a compact, signed token format used for secure authentication
bcrypt	A password hashing algorithm designed to securely store passwords
CRUD	Create, Read, Update, Delete — the four basic operations of persistent storage
REST API	An architectural style for designing networked applications using standard HTTP methods
Axios	HTTP client

Appendix B: Analysis Models

Use Case Diagram - User



Use Case Diagram – Admin

